
React Native

Interview Questions & Answers | 21 questions | Cross-platform native apps with React

Contents

React Native Fundamentals (Q1-Q5)

Components, Styling & Layout (Q6-Q10)

Native Modules, Platform APIs & Navigation (Q11-Q15)

Performance & Debugging (Q16-Q21)

React Native Fundamentals

Q1. What is React Native and how does it differ from React for the web?

React Native is a framework for building native mobile apps using React's component model and JavaScript, but instead of rendering to the browser DOM, it renders to real native UI components (UIView on iOS, android.view on Android). You write components like `<View>` and `<Text>` instead of `<div>` and `<span>`, and styling uses a JavaScript object API (a subset of CSS via Flexbox) rather than CSS files.

Q2. What is the Bridge in React Native (and how does the New Architecture change it)?

In the classic architecture, the Bridge is an asynchronous, batched, serializable (JSON) messaging layer connecting the JavaScript thread to the native side, which can introduce latency for high-frequency interactions like animations or gestures. The New Architecture replaces it with JSI (JavaScript Interface), which allows JavaScript to hold direct references to native objects and call native methods synchronously without serialization, significantly reducing overhead.

Q3. What are the main threads in a React Native app?

A React Native app typically runs several threads.

- JS thread: runs your application logic and React reconciliation
- Main/UI thread: native thread that renders the actual UI and handles gestures
- Shadow/Layout thread: computes Flexbox layout via Yoga off the main thread
- Native modules thread(s): background threads for native module work

Q4. What is Metro and what does it do?

Metro is React Native's JavaScript bundler. It resolves and transforms your app's modules (JS/TS/JSX, assets) into a single bundle, supports fast refresh during development, and handles platform-specific file extensions (e.g. `Component.ios.js` vs `Component.android.js`).

Q5. What is the difference between Expo and the React Native CLI (bare workflow)?

Expo is a framework and set of tools/services built on top of React Native that handles native build configuration for you, offers a large library of pre-built native modules, and supports over-the-air updates - great for getting started fast without touching native code. The bare React Native CLI gives you direct access to the native iOS/Android projects, which is necessary when you need custom native modules or full control, though Expo's 'prebuild'/dev client now bridges much of that gap.

Components, Styling & Layout

Q6. How does styling work in React Native compared to CSS on the web?

Styles are plain JavaScript objects (often created with `StyleSheet.create` for minor performance and validation benefits), using camelCase property names. There's no cascade/inheritance like CSS, no CSS classes, and only a subset of CSS properties is supported. All layout defaults to Flexbox, with `flexDirection` defaulting to 'column' (unlike the web's 'row' default).

Q7. How does Flexbox layout in React Native differ from web Flexbox?

The core concepts (`flexDirection`, `justifyContent`, `alignItems`, `flex`) are the same, powered by the same Yoga layout engine, but the default `flexDirection` is 'column' instead of 'row', and there's no support for some web-only properties like float or CSS grid. Every `<View>` behaves like `display: flex` by default.

Q8. What is the difference between FlatList, SectionList, and ScrollView?

`ScrollView` renders all of its children up front, which is fine for small, fixed content but expensive and memory-heavy for long lists. `FlatList` renders items lazily/virtualized (only what's near the viewport), recycling views as you scroll, making it suitable for long or dynamic lists. `SectionList` is a `FlatList` variant for data grouped into sections with headers.

Q9. How would you optimize the performance of a long FlatList?

Several techniques reduce re-renders and memory pressure in long lists.

- Provide a stable `keyExtractor` instead of relying on index
- Memoize `renderItem` and row components with `React.memo`
- Set `getItemLayout` when row height is fixed, to skip measurement
- Tune `windowSize`, `maxToRenderPerBatch`, and `initialNumToRender`
- Avoid anonymous inline functions/objects passed as props on each render
- Use `removeClippedSubviews` on Android for very long lists

Q10. What is the Pressable component and how does it differ from TouchableOpacity?

`Pressable` is the newer, more flexible API for handling touch interactions, exposing press state so styles/children can respond to pressed, hovered, or focused states via functions. `TouchableOpacity` is an older, simpler API that just dims opacity on press. `Pressable` is generally recommended for new code.

Native Modules, Platform APIs & Navigation

Q11. How do you write a custom Native Module in React Native?

A native module is written in Swift/Objective-C (iOS) or Kotlin/Java (Android), exposing methods to JavaScript through React Native's native module system (TurboModules in the New Architecture, or the legacy bridge module API). It's used when you need functionality not available in JS or existing libraries, such as custom hardware access or performance-critical native code.

Q12. How do you handle platform-specific code in React Native?

Two common approaches: using the `Platform` module to branch logic at runtime (`Platform.OS === 'ios'`), or creating separate files with `.ios.js/.android.js` extensions that Metro automatically resolves per platform, keeping platform-specific implementations cleanly separated.

```
import { Platform, StyleSheet } from 'react-native';
const styles = StyleSheet.create({
  header: {
    paddingTop: Platform.OS === 'ios' ? 44 : 24,
    ...Platform.select({ ios: { shadowOpacity: 0.3 }, android: { elevation: 4 } }),
  },
});
```

Q13. How does navigation typically work in a React Native app?

Most apps use React Navigation (JS-based, highly customizable) or React Native Screens/Navigation libraries backed by native navigation primitives for improved performance. Common navigator types are Stack (push/pop screens), Tab (bottom or top tabs), and Drawer (side menu), which can be nested and combined.

Q14. How do you handle deep linking in React Native?

Deep linking is configured at the native level (URL schemes on iOS Info.plist, intent filters on Android Manifest) and then wired into your navigation library (e.g. React Navigation's linking config) to map incoming URLs to specific screens and params.

Q15. How do you persist data locally in a React Native app?

Common options depending on data size and sensitivity.

- AsyncStorage / @react-native-async-storage - simple key-value storage
- MMKV - much faster key-value storage backed by native code
- SQLite (via expo-sqlite or react-native-sqlite-storage) - relational data
- SecureStore / Keychain (iOS) / Keystore (Android) - sensitive data like tokens

Performance & Debugging

Q16. What causes the JS thread to become a performance bottleneck, and how do you diagnose it?

Heavy synchronous computation, large re-renders, or excessive bridge traffic (in the old architecture) can block the JS thread and cause dropped frames or unresponsive UI. Use the in-app Perf Monitor, Flipper, or the React DevTools Profiler to see frame rates and identify slow renders; move expensive computation off the JS thread with things like InteractionManager, worklets (Reanimated), or native modules.

Q17. How do animations work in React Native, and why use Reanimated over the Animated API?

The built-in Animated API can run animations on the native UI thread via useNativeDriver: true for a subset of properties (transform, opacity), avoiding bridge overhead per frame. Reanimated goes further by letting you write animation and even gesture-handling logic that runs entirely on the UI thread using worklets, enabling smooth, complex, interactive animations that would otherwise jank due to JS thread congestion.

Q18. What is the difference between Debug and Release builds, and why does performance differ?

Debug builds include development-only warnings, hot reloading support, and run JavaScript unoptimized (often without the Hermes bytecode precompilation benefits fully realized), making them noticeably slower. Release builds are minified, use precompiled Hermes bytecode, strip dev-only checks, and are what should always be used to judge real-world performance.

Q19. What is Hermes and why does it matter?

Hermes is a JavaScript engine optimized specifically for React Native, designed to improve app start-up time by precompiling JavaScript to efficient bytecode ahead of time, and to reduce memory usage and app size compared to JSC (JavaScriptCore), especially beneficial on lower-end Android devices.

Q20. How would you reduce a React Native app's bundle size and startup time?

Techniques include enabling Hermes, using ProGuard/R8 (Android), removing unused dependencies and assets, lazy-loading screens/features not needed at launch, optimizing images, enabling the New Architecture where applicable, and analyzing the bundle with tools like source-map-explorer to find and trim large dependencies.

Q21. How do you debug a React Native app effectively?

Common tools include Flipper (network, layout, and native logs inspection), React DevTools for component trees and props/state, the Chrome/Hermes debugger for breakpoints, console.log with remote debugging, and native tools like Xcode Instruments or Android Studio Profiler for deep native-side performance issues.